

Suffix Arrays

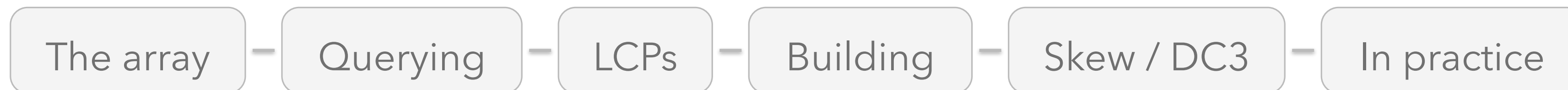
CMSC 423

A suffix tree that is just a sorted list of integers



Where we are

Last lecture we built an index that answers “where does P occur in T?” in $O(n)$ time – and then found it needs about 47 GB for a human genome.



Today: keep the query time, throw the tree away, keep only the order its leaves were in.

structure	space	is P a substring?	status
suffix trie	$\Theta(m^2)$ nodes	$O(n)$	lecture 5
suffix tree	$O(m)$ nodes, ~49 GB	$O(n)$	lecture 5
suffix array	m integers, ~12 GB	$O(n \log m)$	today
...with LCP tables	$3m$ integers, ~37 GB	$O(n + \log m)$	today
FM index	less than m bytes	$O(n)$	next lecture

Notation

- T – the text we index. $m = |T|$, counting the terminator.
- P – the pattern we search for. $n = |P|$.
- Σ – the alphabet. k – the number of occurrences we report.
- $T[i..j]$ is 0-based and inclusive; $T[i:]$ is the suffix starting at i .
- T ends in $\$$, a character smaller than every other and appearing nowhere else – with one exception, the skew algorithm, which pads on the right instead.

Three examples run through this lecture, each where it earns its keep:

$T = \text{abaaba\$}$ $m = 7$ – small enough to draw the whole suffix tree, and the same string as last lecture.

$T = \text{mississippi\$}$ $m = 12$ – big enough for a search tree and a real LCP table.

$T = \text{yabbadabbado}$ $m = 12$, no terminator – DC3 pads instead. The paper's own example.

Watch the letters. m is the text, n is the pattern. A bound of $O(n \log m)$ is “pattern length times log of text length” – the two are not interchangeable, and the papers this lecture cites often swap them.

The suffix array

sorted suffixes, stored as m integers

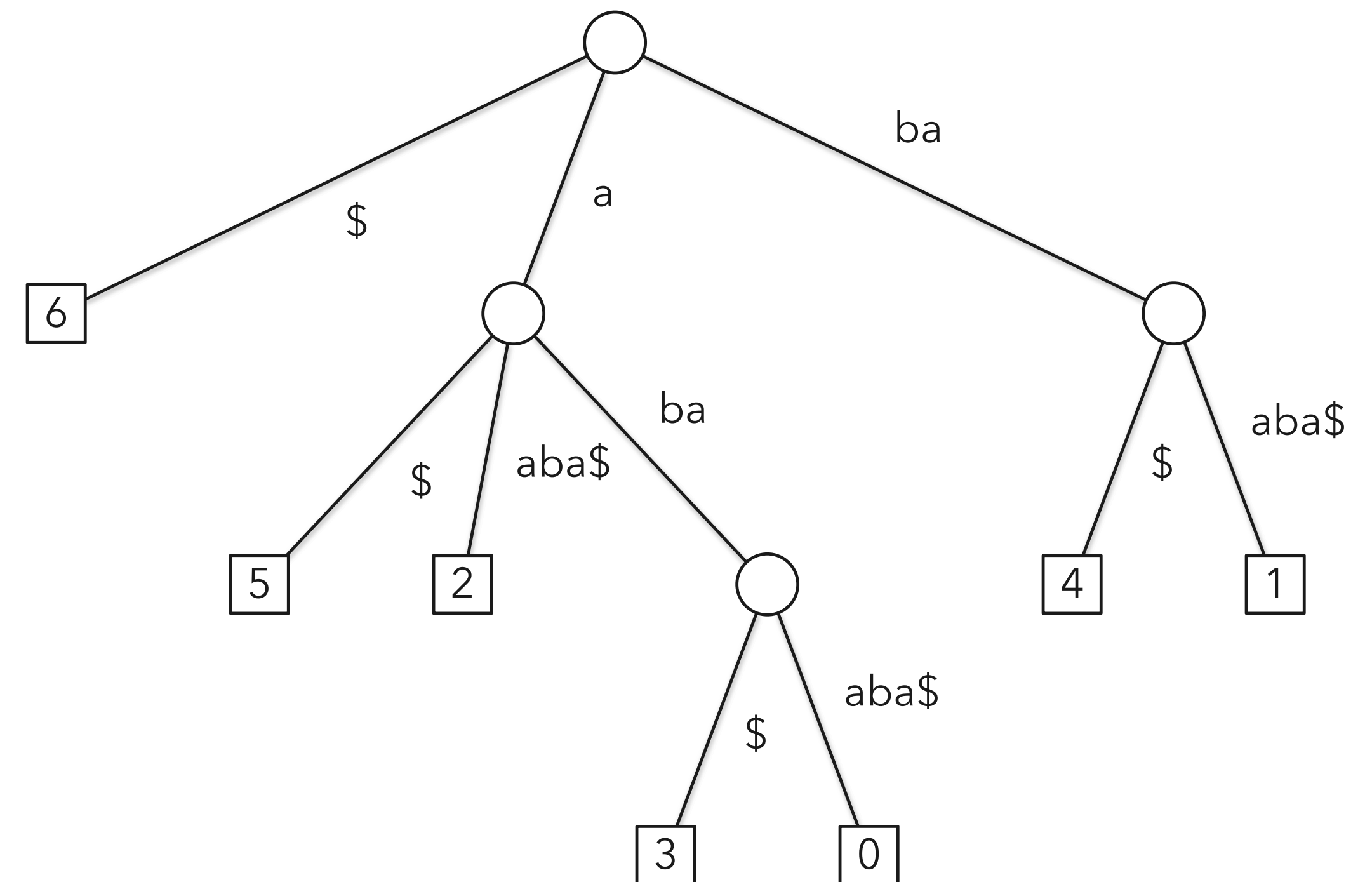


Last lecture: the suffix tree of T

T = abaaba\$

- Every leaf is one suffix of **T**, labelled with the offset it starts at.
- Edge labels are (offset, length) pairs into T, so the tree itself is $O(m)$.
- A query walks down from the root: $O(n)$ time, independent of m .

But: ~15.8 bytes per base in practice. A human genome needs about 49 GB.



The suffix array

SA(T) is the array of the m starting offsets of T's suffixes, listed in lexicographic order of the suffixes they name.

T, then its 7 suffixes

abaaba\$
abaaba\$
baaba\$
aaba\$
aba\$
ba\$
a\$
\$

sort

SA(T)

6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

the offsets, in the order the sort put their suffixes

T itself is part of the index. The array is only m integers – you still need T to know what suffix an offset names.

Building it by hand

Write the suffixes down with their offsets, sort the suffixes, and the offsets come along for the ride. What is left is the suffix array.

offsets, unsorted

0	a b a a b a \$
1	b a a b a \$
2	a a b a \$
3	a b a \$
4	b a \$
5	a \$
6	\$

sort the suffixes



SA(T)

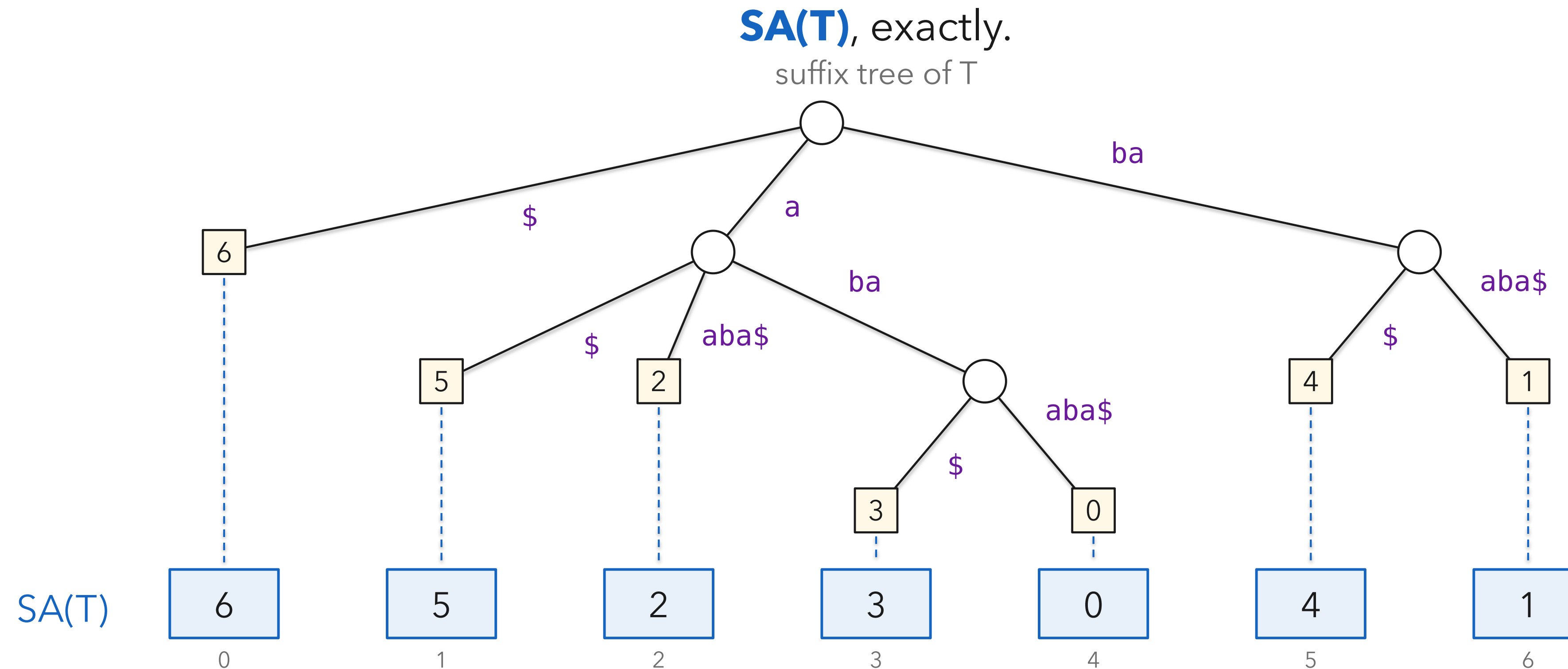
6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

The array is never sorted directly – the suffixes are. The integers are just labels being permuted.

\$ sorts before every other character, so suffix 6 is always first. And because no suffix is a prefix of another, the order is total – there are never ties.

The suffix array is the suffix tree's leaves

Read the leaves of the suffix tree left to right – children in lexicographic order – and you get

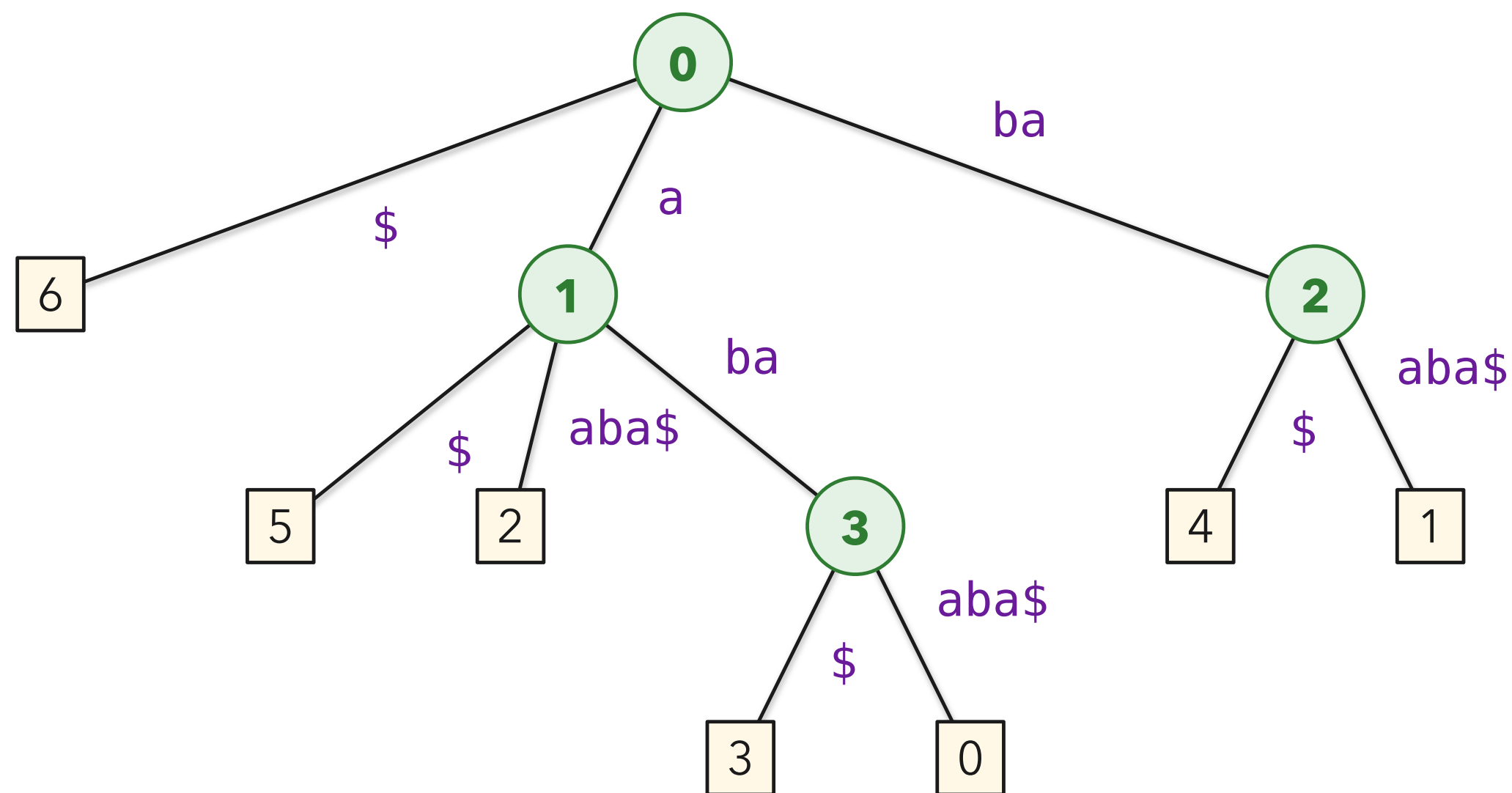


The whole idea of the lecture: the queries only ever used the order the leaves are in. Keep the order, drop the shape, and the index becomes m integers.

...and the LCP array is where the leaves meet

Two adjacent leaves meet at some internal node. The **string-depth** of that node is how much the two suffixes share – their longest common prefix.

string-depths in green



SA(T)	
6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

LCP1(T) = 0 1 1 3 0 2

So SA and LCP1 together carry everything the tree's shape was carrying. That is why the rest of this lecture cares about LCPs at all.

What it costs

Both structures are $O(m)$. The constant factor is the whole story.

index	per base	human genome (3.1 Gbp)	query
suffix tree	~15.8 bytes	~49 GB	$O(n)$
suffix array	4 bytes (one 32-bit int)	~12 GB	$O(n \log m)$
suffix array + LCP tables	12 bytes	~37 GB	$O(n + \log m)$

- A 32-bit integer addresses 4.3×10^9 positions, which is enough for a human genome – just.
- So the suffix array is roughly a quarter of the suffix tree, and it fits in memory on an ordinary machine.
- The price is in the third column: the array has lost the tree's shape, so a query can no longer just walk down. That is the problem the next two sections solve.

Querying

binary search, and how to stop re-reading the same characters



Searching a sorted list of suffixes

- **1.** P occurs in T if and only if P is a prefix of at least one suffix of T.
- **2.** The suffixes that have P as a prefix are **consecutive** in SA(T).

Fact 1 turns “does P occur?” into “is P a prefix of something in this list?”. Fact 2 turns that into “find where P belongs in a sorted list” – which is binary search.

SA(T)

6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

the suffixes
starting with
aba

P = aba occurs at offsets 3 and 0 – rows 3 and 4 of the array, side by side.

Why they are consecutive

Sorting groups equal prefixes together: everything beginning with a is one block, and inside it everything beginning with ab is a smaller block.

SA(T)	
6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

prefix a

prefix ab

So the set of matches is always an interval $[lo, hi)$ of the array – never a scattered set. Two binary searches find its two ends, and $hi - lo$ is the number of occurrences.

Binary search: the invariant

Assume P contains no $\$$, so P is never equal to a suffix. Keep three indices and one invariant:

$$SA[l] < P < SA[r]$$

- l starts at 0 – $SA[0]$ is the suffix $\$$, which is smaller than any P .
- r starts at m – one past the end, standing for “larger than everything”.
- c is the midpoint $\lfloor (l + r) / 2 \rfloor$, the entry we compare against this round.

Writing $SA[l]$ we mean the suffix it names, $T[SA[l]:]$ – spelling that out every time is unreadable.

Because $SA[l] < P < SA[r]$ always holds, the answer – the first row where P could be inserted – is always strictly between l and r . The search ends when there is no room left between them.

One iteration

- Let $c = \lfloor (l + r) / 2 \rfloor$ and compare P against the suffix $SA[c]$, character by character.
- If $P < SA[c]$ the answer is in the top half: set $r = c$.
- If $P > SA[c]$ the answer is in the bottom half: set $l = c$.

When to stop

- $P < SA[c]$ and $c = l + 1 \Rightarrow$ the answer is c
- $P > SA[c]$ and $c = r - 1 \Rightarrow$ the answer is r

In both cases the interval has closed: no index strictly between l and r is left to test, so the insertion point is forced.

SA(T)	
l	6 \$
	5 a \$
	2 a a b a \$
c	3 a b a \$
	0 a b a a b a \$
	4 b a \$
	1 b a a b a \$

$r = 7$ – there is no row 7; it stands for “larger than everything”.

Worked example: search $T = \text{abaaba}\$$ for $P = \text{aba}$

Step 1

$$l = 0 \quad r = 7 \quad c = \lfloor (0 + 7) / 2 \rfloor = 3$$

$SA[c] = 3$, the suffix **aba**\$

aba < **aba**\$

so set $r = c = 3$

aba is a prefix of aba\$, and a prefix is always the smaller string.

SA(T)	
l	6 \$
	5 a \$
	2 a a b a \$
c	3 a b a \$
	0 a b a a b a \$
	4 b a \$
	1 b a a b a \$

$r = 7$, just past the end

after this step, the answer must lie strictly between l and r :

l		c		r			
6	5	2	3	0	4	1	
0	1	2	3	4	5	6	

Worked example: search $T = \text{abaaba}\$$ for $P = \text{aba}$

Step 2

$$l = 0 \quad r = 3 \quad c = \lfloor (0 + 3) / 2 \rfloor = 1$$

$SA[c] = 5$, the suffix **a\$**

aba > **a\$**

so set **$l = c = 1$**

aba and a\$ agree on 1 character, then differ at position 1: 'b' against '\$'.

SA(T)	
<i>l</i>	6 \$
<i>c</i>	5 a \$
	2 a a b a \$
<i>r</i>	3 a b a \$
	0 a b a a b a \$
	4 b a \$
	1 b a a b a \$

after this step, the answer must lie strictly between l and r :

	<i>l</i>	<i>c</i>	<i>r</i>			
6	5	2	3	0	4	1
0	1	2	3	4	5	6

Worked example: search $T = \text{abaaba}\$$ for $P = \text{aba}$

Step 3

$$l = 1 \quad r = 3 \quad c = \lfloor (1 + 3) / 2 \rfloor = 2$$

$SA[c] = 2$, the suffix **aaba**\$

aba > **aaba**\$

so set $l = c = 2$

aba and aaba\$ agree on 1 character, then differ at position 1: 'b' against 'a'.

SA(T)	
6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

after this step, the answer must lie strictly between l and r :

		l	r			
6	5	2	3	0	4	1
0	1	2	3	4	5	6

Worked example: search $T = \text{abaaba}\$$ for $P = \text{aba}$

Done

- $c = r - 1$ and $P > SA[c]$, so the search stops and the answer is $r = 3$.
- Row 3 is the first row whose suffix starts with aba.
- A second search – for the smallest string larger than every aba... – gives the other end, 5.
- **So the occurrences are rows [3, 5) of SA: offsets 3 and 0.**

a b a a b a \$
0 1 2 3 4 5 6

└──────────┘ at 3

└──────────┘ at 0

SA(T)	
6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

all matches

When P does not occur

Nothing changes. The search still ends at an insertion point – it is just an empty one.

interval	midpoint	suffix SA[c]	comparison	action
$l = 0, r = 7$	$c = 3$	aba\$	abb > aba\$	$l = c$
$l = 3, r = 7$	$c = 5$	ba\$	abb < ba\$	$r = c$
$l = 3, r = 5$	$c = 4$	abaaba\$	abb > abaaba\$	$l = c$

The search returns row **5**. The range of matches is $[5, 5)$ – empty – so **abb** does not occur in T.

This is the same test for both questions. “Does P occur?” is “is the range non-empty?”, and “how many times?” is “how wide is it?”.

One algorithm, two answers.

Counting and reporting occurrences

- Search for P – this gives **lo**, the first row whose suffix has P as a prefix.
- Search again for the first row past all of them – this gives **hi**.
- Then **hi – lo** is the number of occurrences, and SA[lo..hi-1] are the offsets.

query	answer for T = abaaba\$
is aba a substring?	yes – the range [3, 5) is not empty
how many times does aba occur?	$5 - 3 = 2$
where?	SA[3] = 3 and SA[4] = 0
is abb a substring?	no – the range [5, 5) is empty

SA(T)

6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

lo = 3
 hi = 5
 2 occurrences

Reporting k matches costs $O(k)$ once the range is known – they are already sitting next to each other.

What does that cost?

- The interval halves each round, so there are $\lceil \log_2 m \rceil$ iterations.
- Each iteration compares P against one suffix, which takes up to $O(n)$ character comparisons.

$O(n \log m)$

	suffix tree	suffix array
is P a substring?	$O(n)$	$O(n \log m)$
report k occurrences	$O(n + k)$	$O(n \log m + k)$
space	~49 GB	~12 GB

We traded a factor of $\log m$ in time for a factor of 4 in space. The rest of the lecture wins the time back without giving the space away.

Where the wasted work is

Every iteration starts comparing at character 0 – even though we already know a great deal about how P relates to the suffixes at l and r .



If P and $SA[l]$ share their first 3 characters, and $SA[c]$ lies between them in sorted order, then $SA[c]$ must share those 3 characters too. Nothing in the first 3 positions can possibly differ – so do not look at them.

Making that precise is what the next section is about.

Longest common prefixes

the bookkeeping that removes the $\log m$ factor



Longest common prefix

LCP(S, T) is the longest string that is a prefix of both – everything they share before the first difference.

S	ACTTACAGACGAC	CCGAGAC
T	ACTTACAGACGAC	GGAGCTAGC

first difference

LCP(S, T) = **ACTTACAGACGAC**, of length 13

From here on, LCP(S, T) means the length, not the string. We will only ever need the number.

Two suffixes of the same T can share a long prefix – in a genome, very long. That is exactly the case where naive binary search wastes the most work.

The inequality that makes it work

Suppose we already know how much P shares with the suffixes at l and at r. Then we get a free lower bound on how much it shares with the one at c.

$$\text{LCP}(\mathbf{P}, \mathbf{SA}[c]) \geq \min (\text{LCP}(\mathbf{P}, \mathbf{SA}[l]), \text{LCP}(\mathbf{P}, \mathbf{SA}[r]))$$

- Why: SA[c] sits between SA[l] and SA[r] in sorted order. If SA[l] and SA[r] both agree with P on their first j characters, every suffix ordered between them must agree on those j characters too – there is nowhere else for it to sit.
- So the comparison at c can skip the first min(·, ·) characters and start there.

Note what this costs: nothing. The two LCP values on the right were computed by earlier iterations of this same search. We are only remembering them.

Gusfield calls the search that does this the “simple accelerator”.

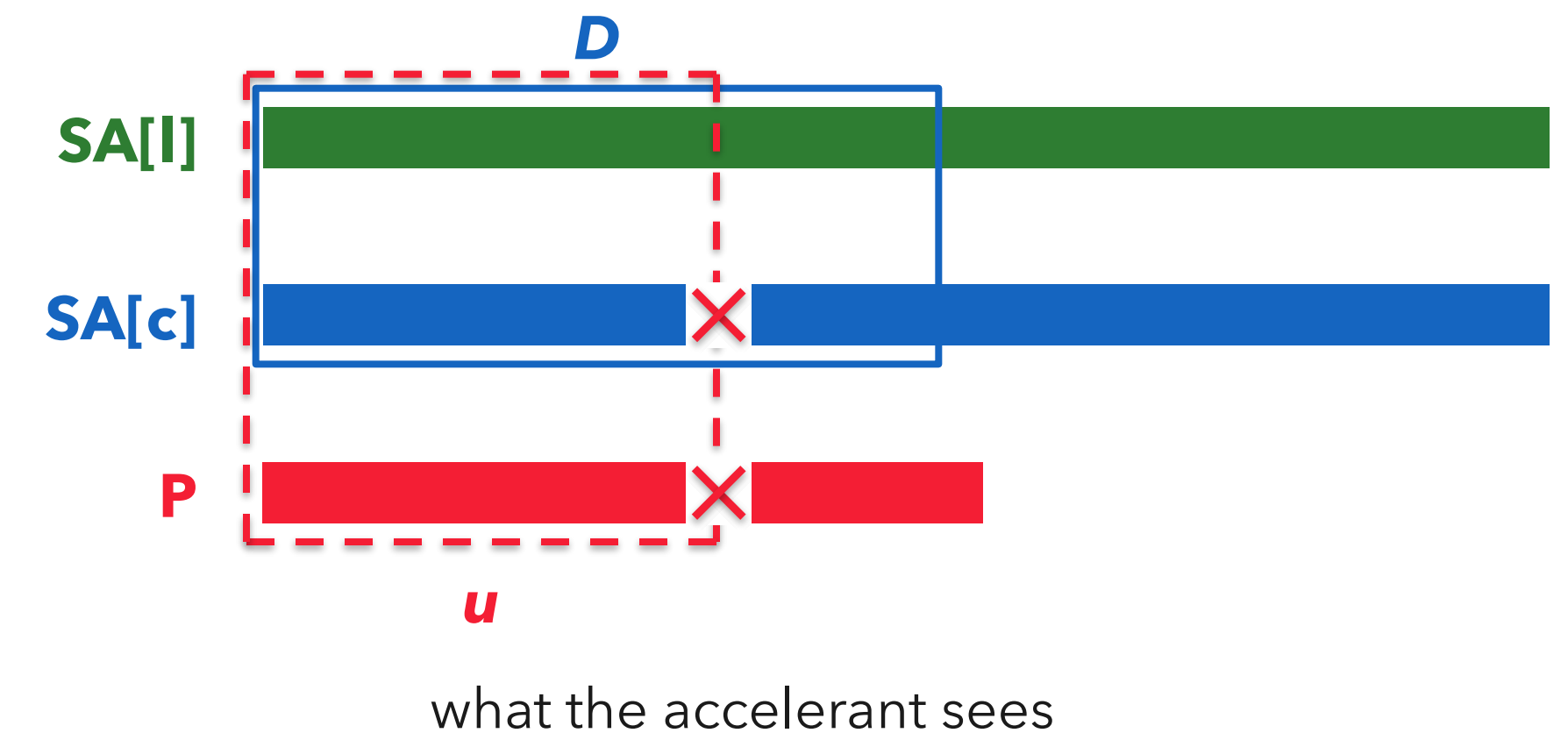
The simple accelerant

- Carry two numbers through the search: how much P shares with SA[l], and how much with SA[r]. Both start at 0.
- At each iteration, begin the character comparison at offset $\min(\text{lcp}_l, \text{lcp}_r)$ instead of 0.
- Whichever side you bisect to, the number of characters you just matched becomes the new lcp for that side.
- Cheap: two extra integers, no preprocessing at all – and on real text a large win.

But the worst case has not moved

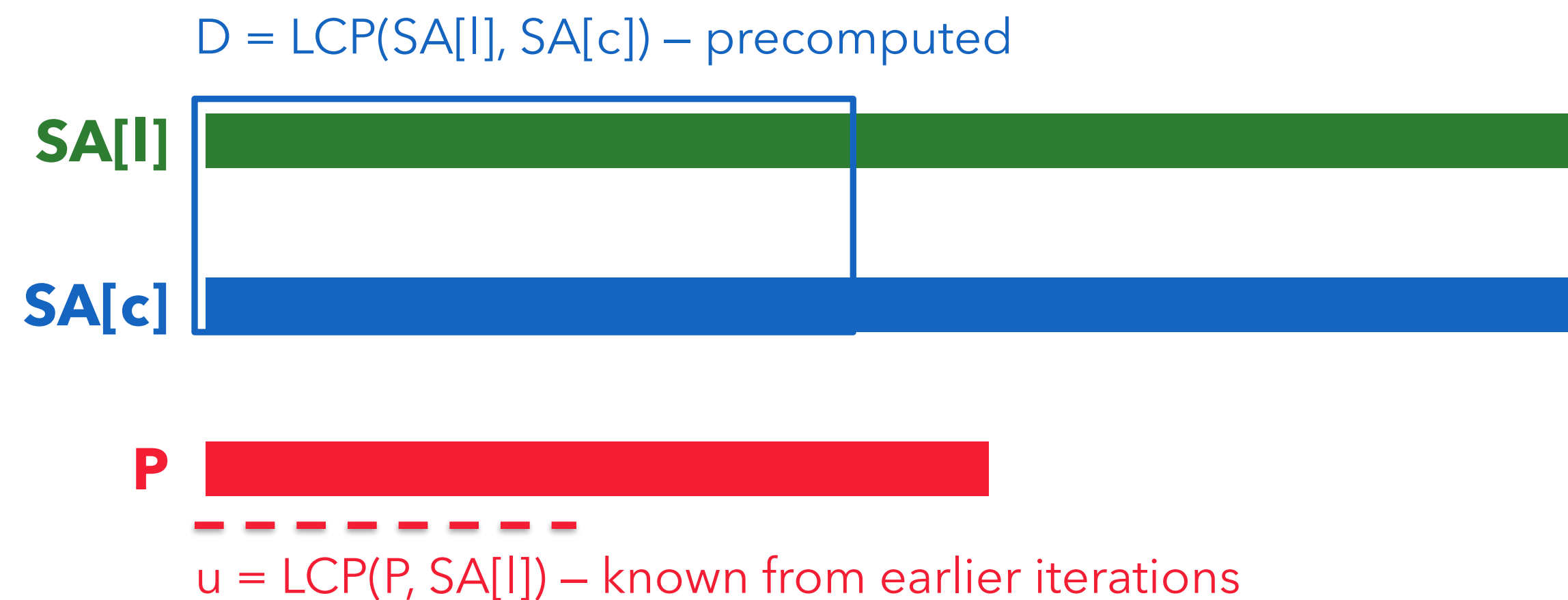
$$T = ac^{m-3}b\$ \quad P = c^{m-3}$$

r never moves off the \$ suffix, so lcp_r stays 0 – and the accelerant starts from $\min(\text{lcp}_l, \text{lcp}_r)$, which is therefore 0 every single time. Each of the $\log m$ iterations rescans nearly all of P, so the worst case is still **$O(n \log m)$** .



The super accelerant

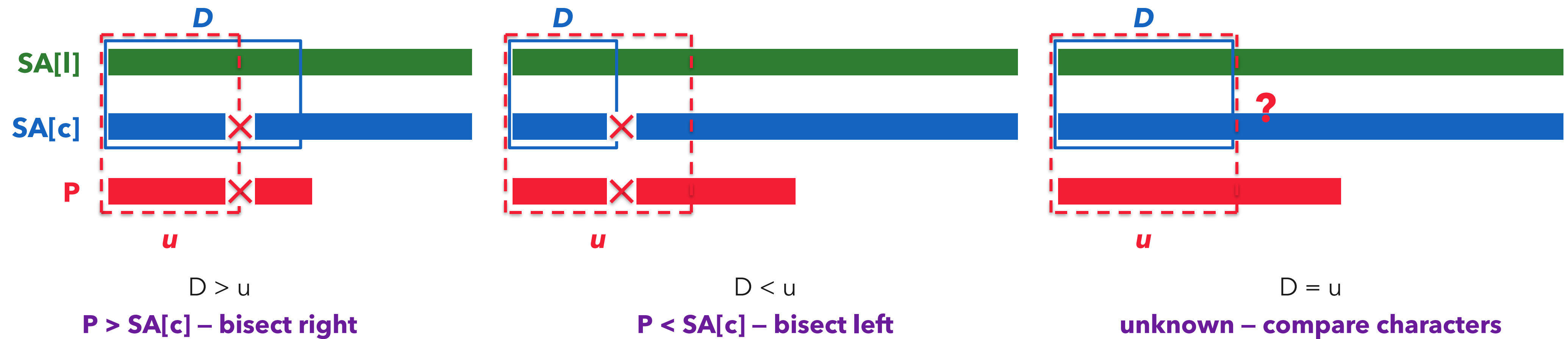
The missing ingredient is the LCPs among T's own suffixes – how much $SA[l]$ and $SA[c]$ share, independently of P. Those can be precomputed once.



- u is already in hand – the simple accelerant was carrying it.
- D is a table lookup, in constant time.
- Comparing u against D decides the whole iteration, often without looking at a single character of P .

Three cases

Write $D = \text{LCP}(\text{SA}[l], \text{SA}[c])$ and $u = \text{LCP}(P, \text{SA}[l])$. Exactly one of three things is true, and each one tells us what to do.

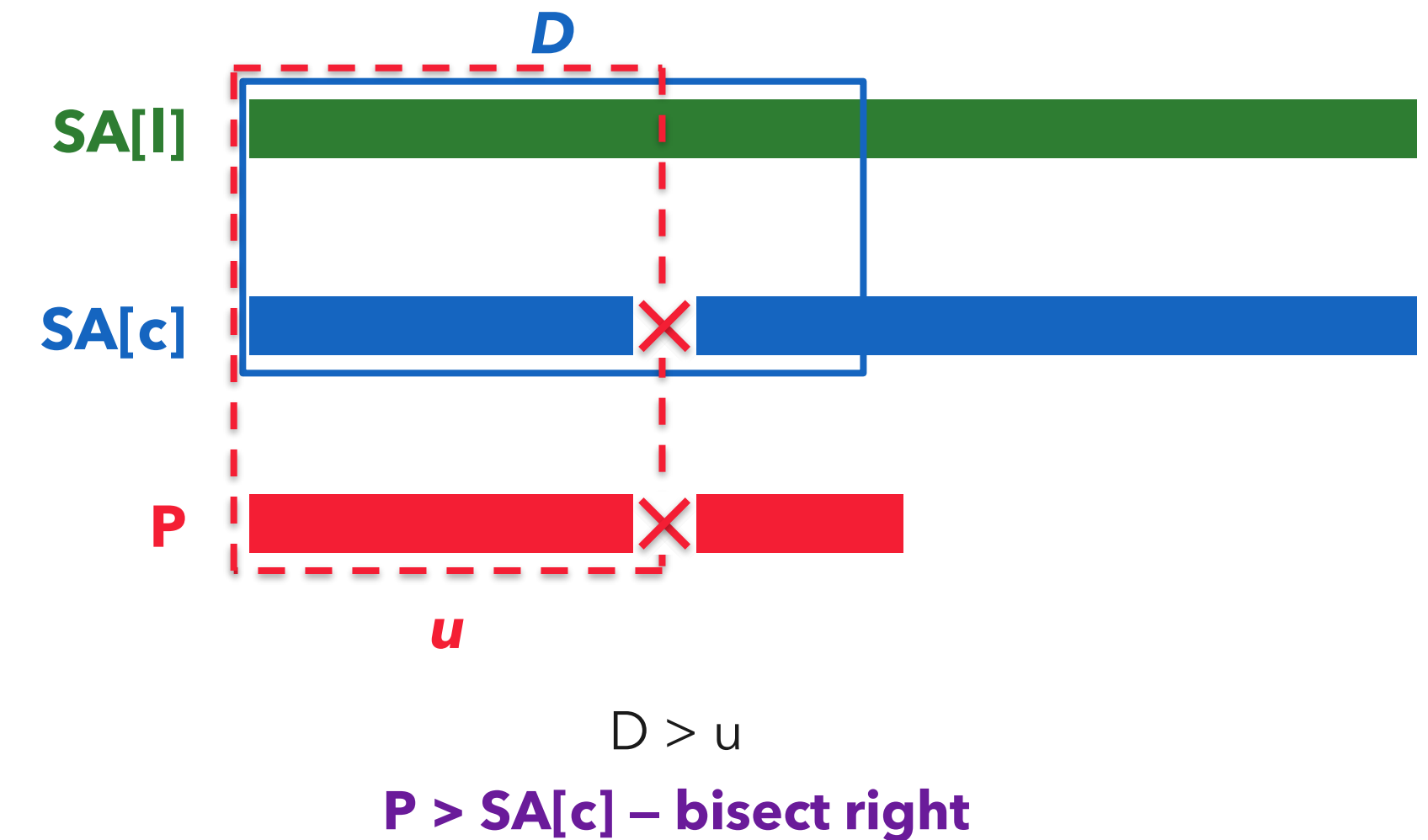


In two of the three cases the iteration costs $O(1)$: we learn which way to bisect from two integers. Only the third case touches P at all.

Case 1: $D > u$

- $SA[l]$ and $SA[c]$ agree further than P and $SA[l]$ do.
- So at position u – where P first differs from $SA[l]$ – $SA[c]$ still matches $SA[l]$, and P does not.
- P 's character there must be the larger one, because P sorts after $SA[l]$.
- Therefore **$P > SA[c]$** : set $l = c$ and look in the lower half.
if $D > u$:
 $l = c$

No character of P was examined.

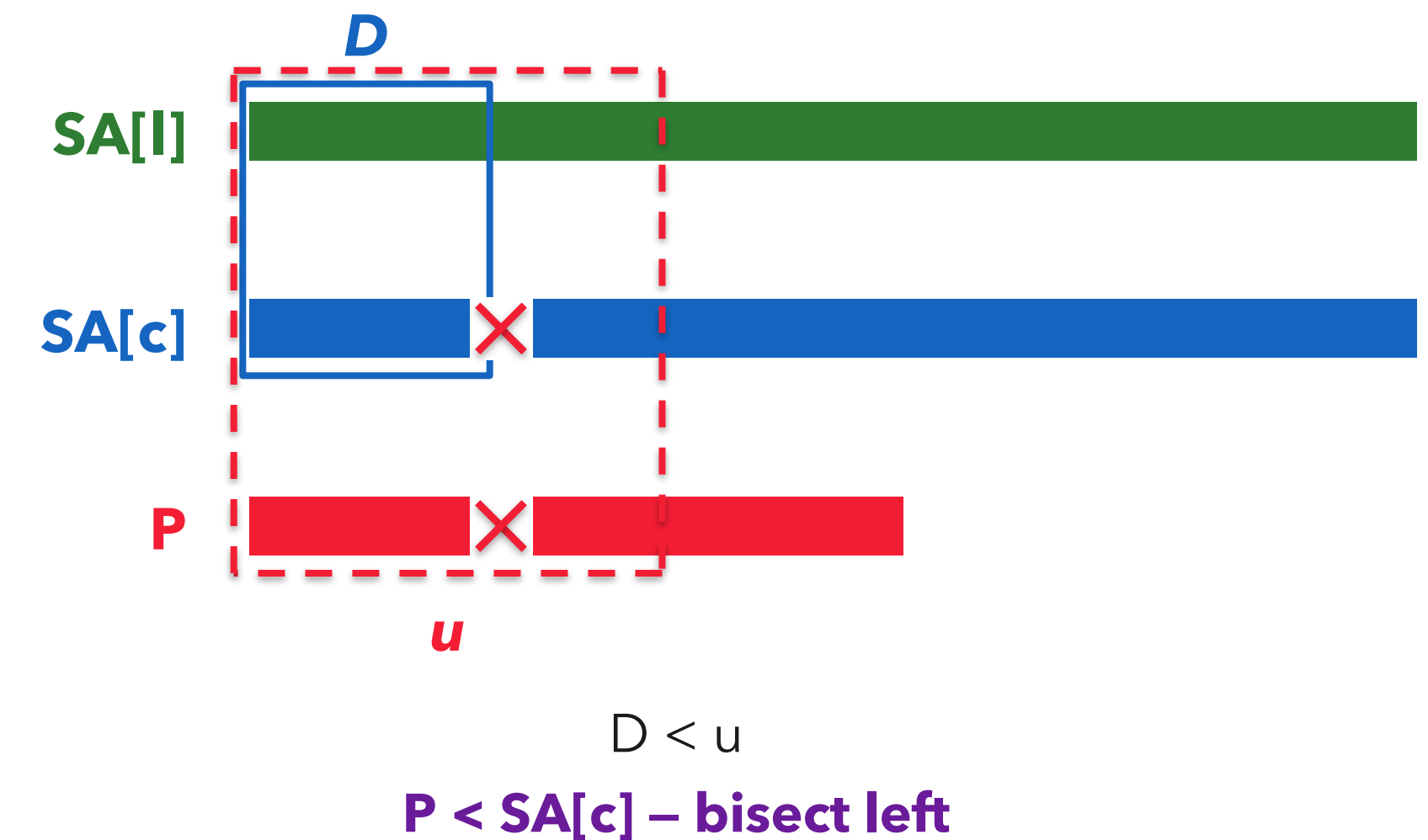


Case 2: $D < u$

- P and SA[l] agree further than SA[l] and SA[c] do.
- So at position D, SA[c] differs from SA[l] while P still matches it.
- SA[c]'s character there must be the larger one, because SA[c] sorts after SA[l].
- Therefore **$P < SA[c]$** : set $r = c$ and look in the upper half.

if $D < u$:
 $r = c$

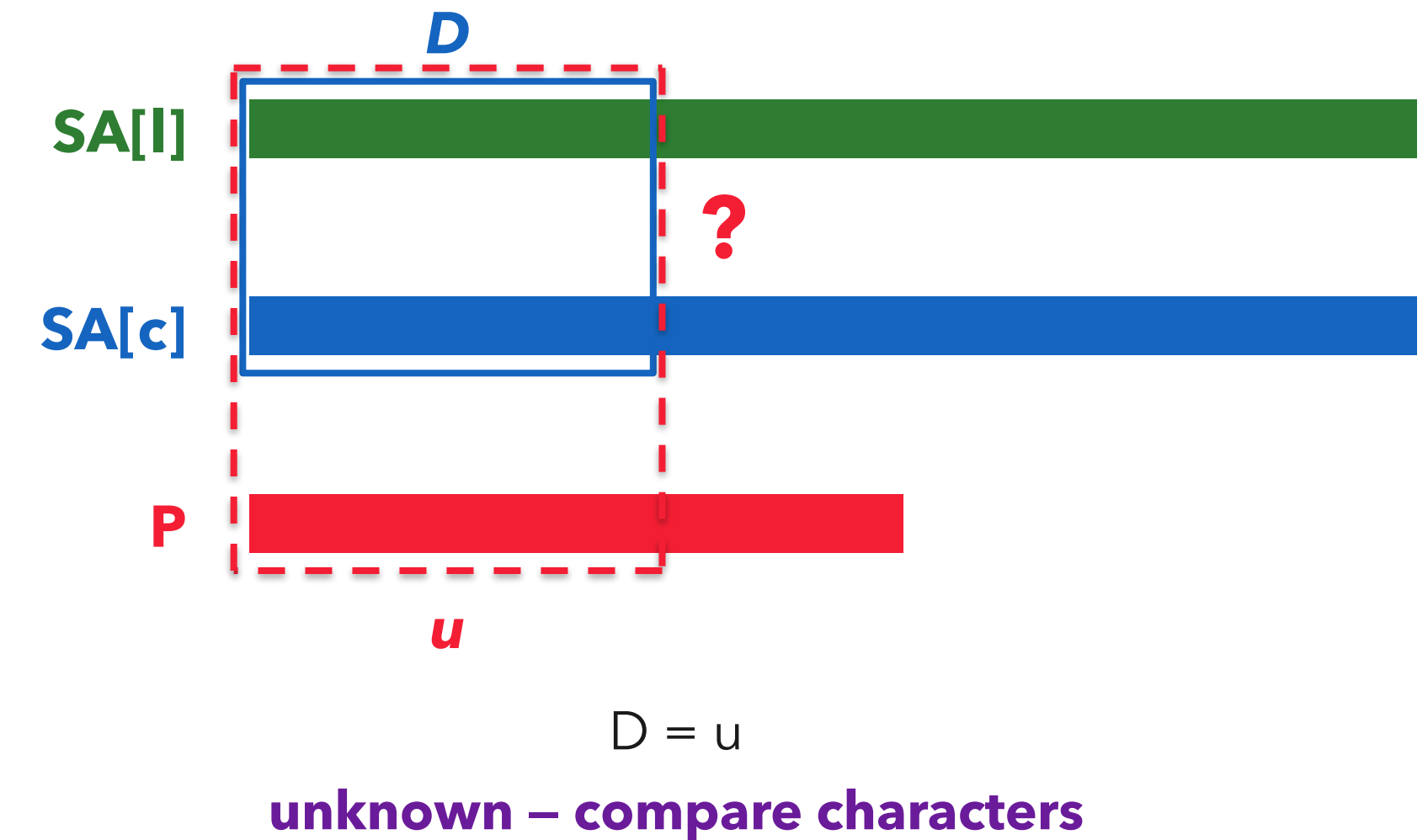
Again, no character of P was examined.



Case 3: $D = u$

- P and SA[c] agree with SA[l] for exactly the same distance, so they agree with each other for at least that far.
- Beyond that, we genuinely do not know – so compare P and SA[c] character by character, **starting at offset u**.
- Every character that **matches** extends the LCP we carry forward, and is never compared again.
- The first character that **mismatches** decides the bisection, and ends the iteration.
else:
 $i = u$
 while $i < n$ and $P[i] == T[SA[c]+i]$:
 $i += 1$

This is the only case that costs more than $O(1)$ – and it pays for itself.



Running time

Theorem. Given the LCP values among T's suffixes, searching for P in a suffix array of size m takes **$O(n + \log m)$** time.

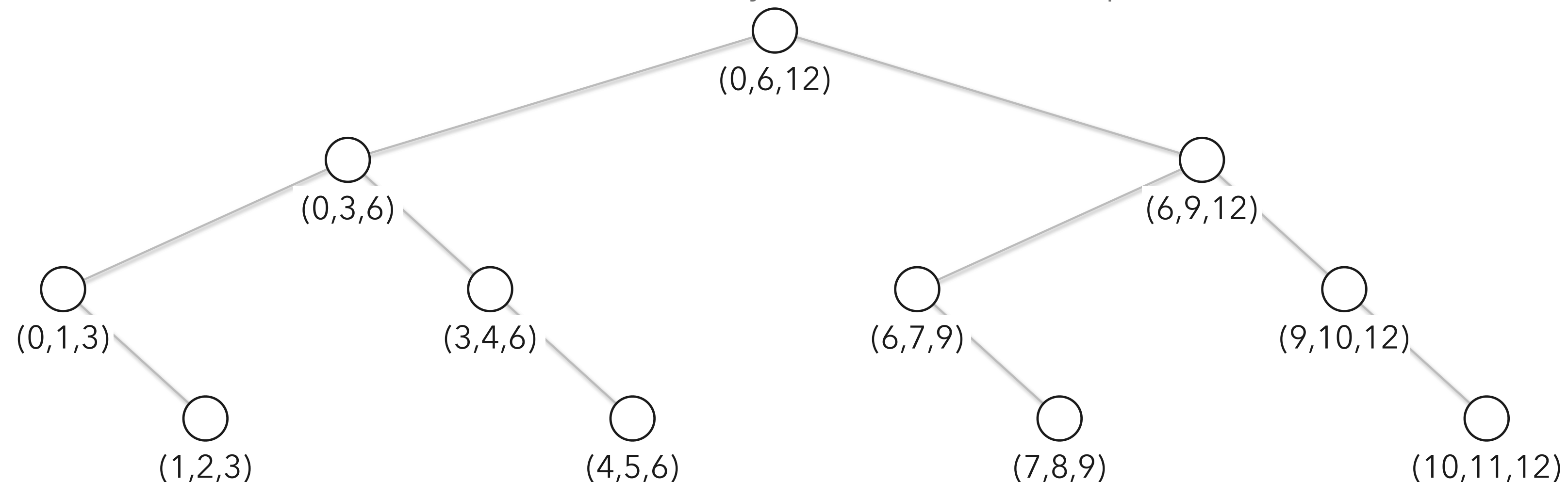
- In cases 1 and 2 the iteration makes $O(1)$ comparisons, and there are at most $\lceil \log_2 m \rceil$ bisections in total.
- In case 3 we compare characters starting at some offset. Each character that matches extends the running LCP, and that offset never decreases – so a matching character is compared **once in the whole search**, giving at most $O(n)$ of them.
- A mismatching character may be re-compared, but each mismatch ends an iteration, so there are at most $\lceil \log_2 m \rceil$ of those.
- Total: $O(n)$ matches + $O(\log m)$ mismatches + $O(\log m)$ bisections = **$O(n + \log m)$** . $\square$

Compare the suffix tree's $O(n)$. We are paying an additive $\log m$ instead of a multiplicative one – for a human genome, $\log_2 m \approx 32$, against patterns that are typically 100 characters long.

Which LCPs do we actually need?

Not all m^2 of them. Binary search only ever visits the midpoints of a fixed recursion – the same (l, c, r) triples for every query – so only those LCPs are ever looked up.

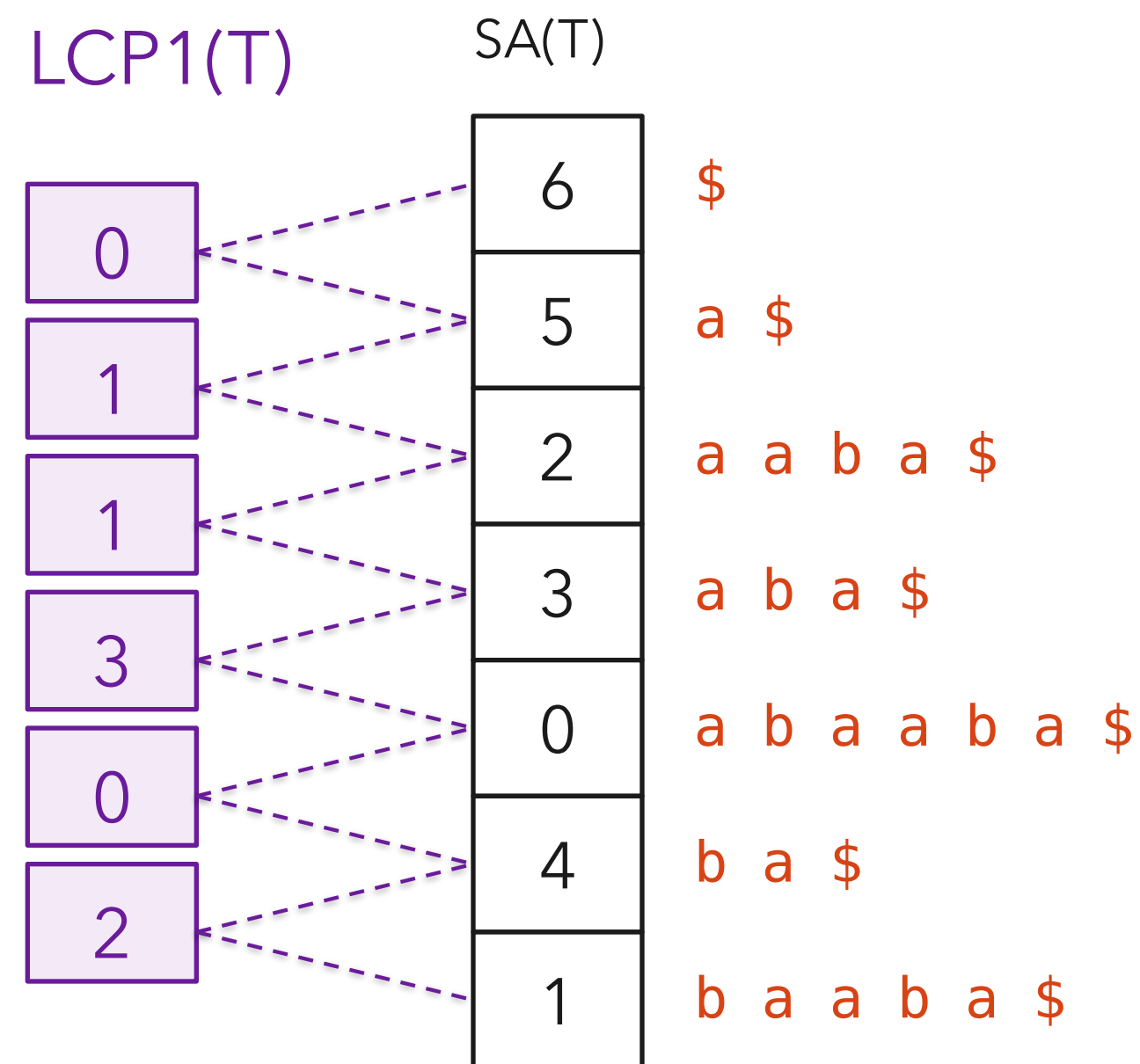
the search tree for $m = 12$: every node is one (l, c, r) triple



- There are exactly $m - 1$ such triples – one per node of the search tree.
- For each we need two numbers: $\text{LCP}(\text{SA}[l], \text{SA}[c])$ and $\text{LCP}(\text{SA}[c], \text{SA}[r])$.
- That is $2m$ integers – the same order as the array itself, not its square.

The LCP1 array

$\text{LCP1}[i] = \text{LCP}(\text{SA}[i], \text{SA}[i+1])$ – how much each suffix shares with the one directly below it. It has $m - 1$ entries.



Each entry looks at one adjacent pair – the two dashed lines feeding it.
LCP1 is the building block: every other LCP we need is a minimum over a run of LCP1 entries.

The LCP of a whole range

The LCP of any set of consecutive suffixes is the minimum LCP1 over the adjacent pairs inside it.

$$\text{LCP}(\text{SA}[i], \text{SA}[i+1], \dots, \text{SA}[j]) = \mathbf{\min} (\text{LCP1}[i], \text{LCP1}[i+1], \dots, \text{LCP1}[j-1])$$

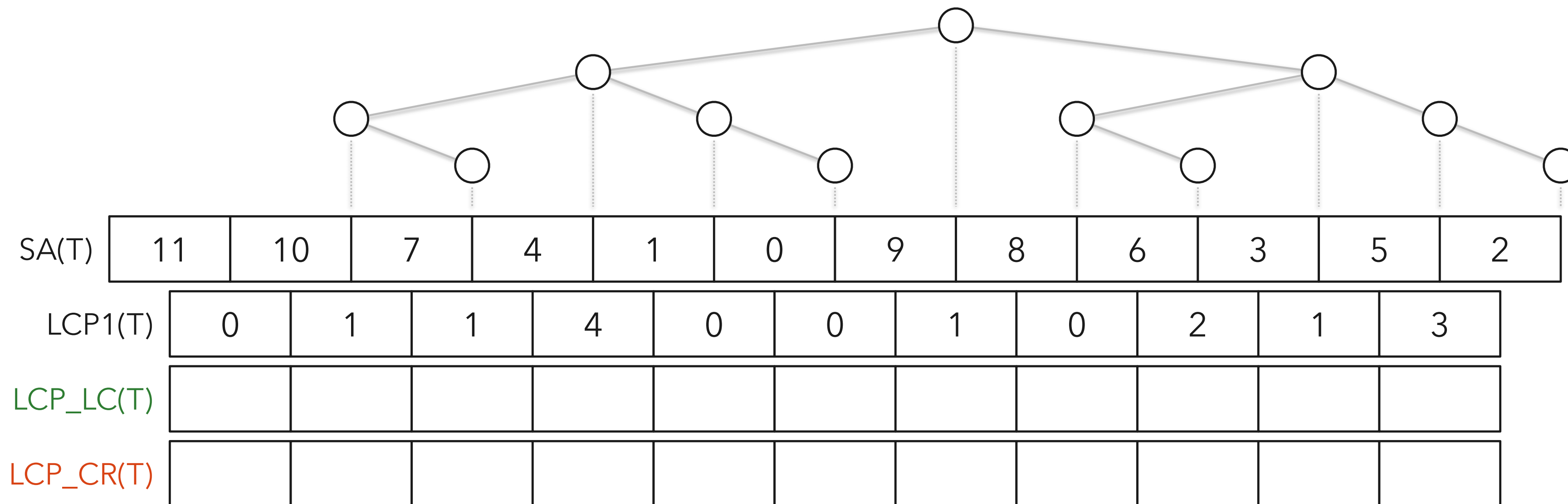
- Why: the suffixes are sorted, so as you walk down the block the shared prefix can only get shorter and then stay shorter. The narrowest point in the middle is the whole block's agreement.
- So one array of $m - 1$ numbers answers every range-LCP question we will ask.

LCP1(T)	0	1	1	3	0	2
	0	1	2	3	4	5

$\text{LCP}(\text{SA}[2], \text{SA}[3], \text{SA}[4]) = \min(\mathbf{1}, \mathbf{3}) = \mathbf{1}$ – and indeed **a**aba\$ and **a**baaba\$ share the single character **a**.

Pre-computing the two LCP tables

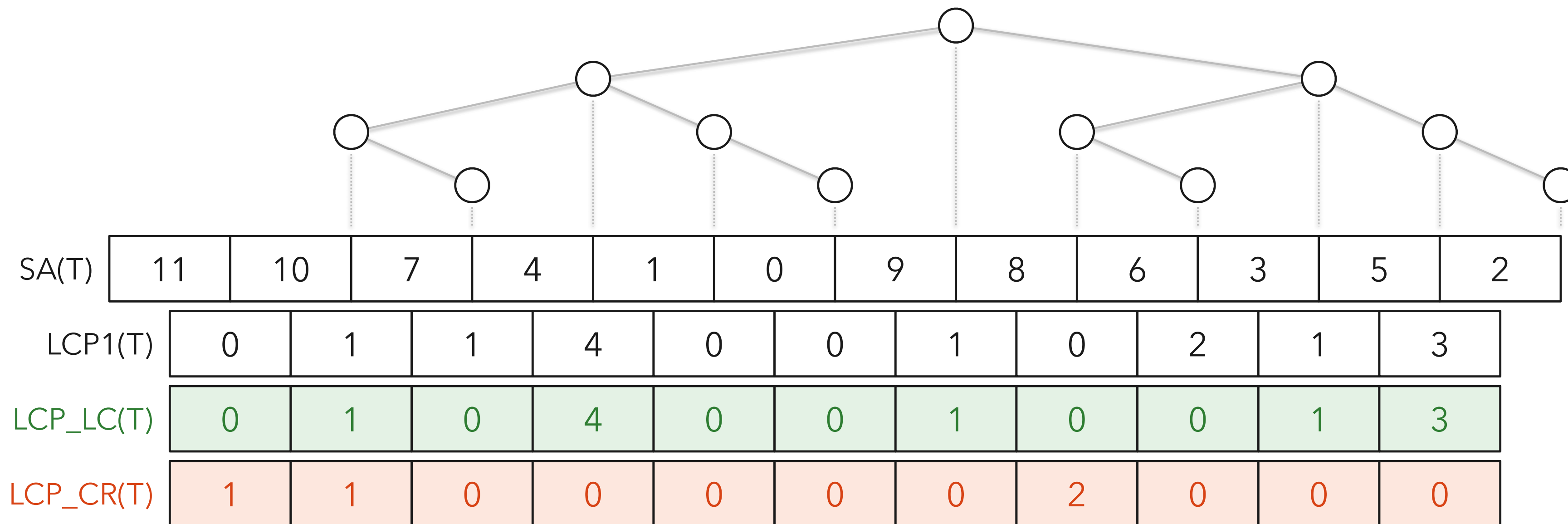
One bottom-up pass over the search tree. At a leaf, each value is an LCP1 entry; at an internal node, LCP_LC is the smaller of the LEFT child's two values and LCP_CR the smaller of the RIGHT child's.



T = mississippi\$ Each search-tree node (l, c, r) stores its two values at index $c - 1$.

Pre-computing the two LCP tables

Filled in: $\text{LCP_LC}[c-1] = \text{LCP}(\text{SA}[l], \text{SA}[c])$ and $\text{LCP_CR}[c-1] = \text{LCP}(\text{SA}[c], \text{SA}[r])$ for every node of the search tree.



T = mississippi\$ Time and space: **O(m)** – one pass, two arrays of $m - 1$ integers.

One indexing trap

This is the detail that breaks implementations.

The node $(l, c, r) = (0, 3, 6)$ stores its values at index $c - 1 = 2$, not at index c .

$\text{LCP}(\text{SA}[l], \text{SA}[c])$ is **LCP_LC[c - 1]** $\text{LCP}(\text{SA}[c], \text{SA}[r])$ is **LCP_CR[c - 1]**

The reason: the tables have $m - 1$ entries, one per adjacent pair, while the suffix array has m . There is no entry 0 to hold a node whose c is 0 – and there is no such node, because every c is strictly between its l and its r .

A worked lookup

you want	you read	value for mississippi\$
$\text{LCP}(\text{SA}[0], \text{SA}[3])$	$\text{LCP_LC}[2]$	0
$\text{LCP}(\text{SA}[3], \text{SA}[6])$	$\text{LCP_CR}[2]$	0

The three ways to search, compared

method	extra space	worst case	in practice	Gusfield's name
plain binary search	–	$O(n \log m)$	$O(n \log m)$	–
skip using LCPs with P	–	$O(n \log m)$	$\approx O(n + \log m)$	simple accelerant
skip using all LCPs	2m integers	$O(n + \log m)$	$O(n + \log m)$	super accelerant

The middle row is the one people actually ship: no preprocessing, no extra memory, and in practice almost all of the benefit.

Measured

500K 100-nt patterns matched against the ~5 Mnt E. coli genome. Index building time not included.

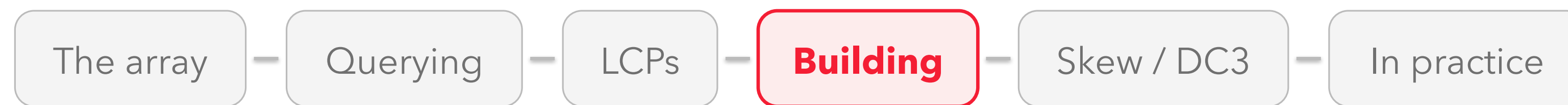
	super accelerant	simple accelerant	no accelerant
python -O	68.78 s	69.80 s	102.71 s
pypy -O	5.37 s	5.21 s	8.74 s
character comparisons	99.5 M	117 M	235 M

Read the second row carefully: under PyPy the simple accelerant beats the super accelerant, even though it does 18 % more character comparisons. The extra 2m integers cost cache misses, and on this workload that costs more than the comparisons it saves.

Asymptotics choose the algorithm; the memory hierarchy picks the winner.

Building the suffix array

from the tree, and then without one



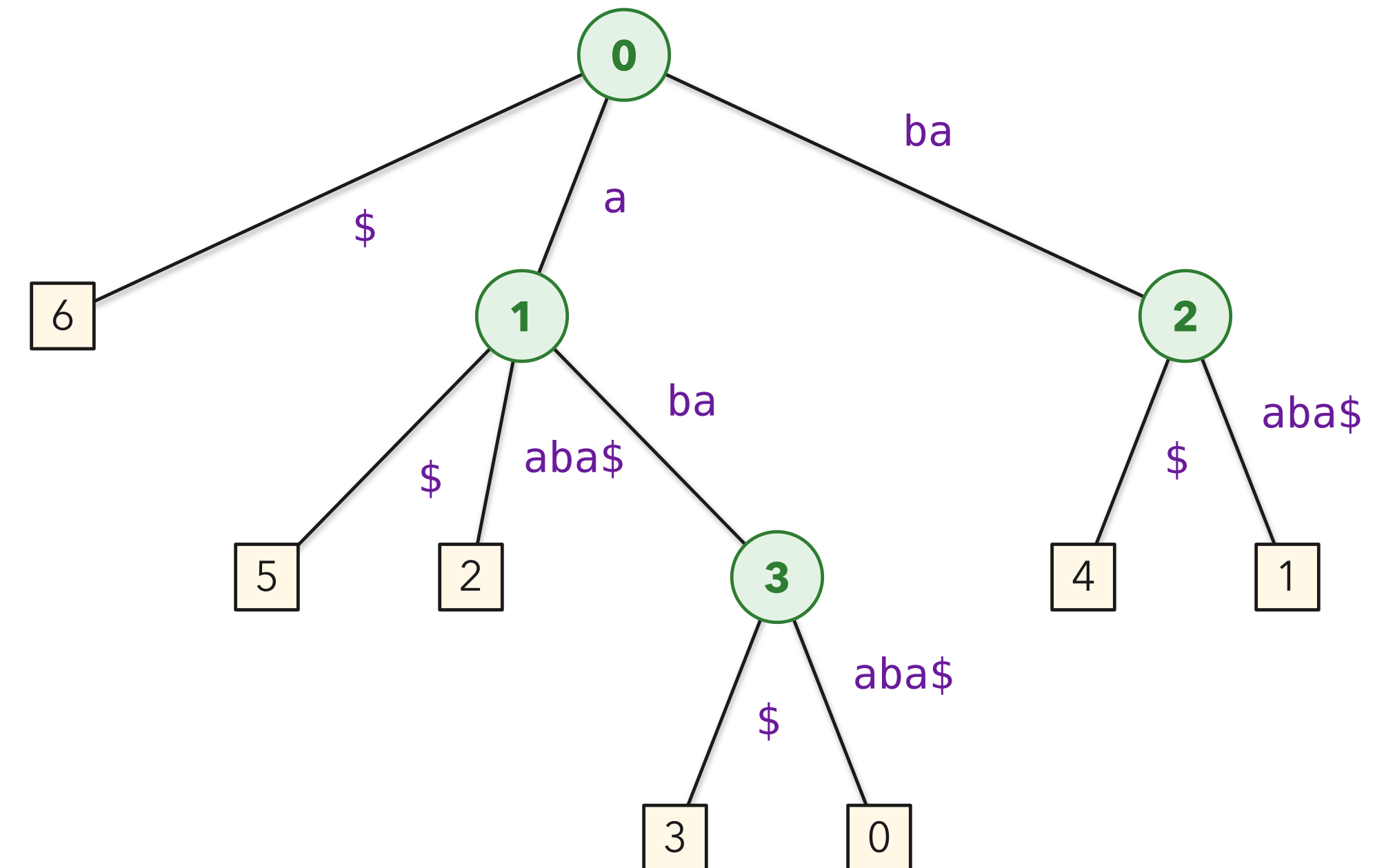
From the suffix tree

- Do a depth-first traversal, visiting each node's children in lexicographic order, and emit each leaf's offset as you reach it.
- That is exactly the leaf order – which is exactly $SA(T)$.
- $O(m)$ nodes visited, m integers emitted, so **$O(m)$** time, assuming the edges are already ordered.

And LCP1 comes free on the same walk

- Between two consecutive leaves the walk goes up to their meeting node and back down.
- Record the **shallowest string-depth** seen on that trip – that is their LCP.

This is the correspondence from the start of the lecture, used as an algorithm: the tree's leaf order is SA, and the depths it passes through are LCP1.



string-depths in green – read them off in leaf order and you have LCP1

But suffix trees are big

Building SA from a suffix tree means building a suffix tree – and the 49 GB we were trying to avoid comes straight back. Can we sort the suffixes directly?

approach	time	why
your favourite comparison sort (quicksort)	$O(m^2 \log m)$	$O(m \log m)$ comparisons, but each compares two suffixes, which is $O(m)$
a sort that knows the items are strings (multikey quicksort)	$O(m^2)$	no character is re-examined within a comparison, but suffixes still overlap
a sort that knows the items are suffixes of the same string	$O(m \log m)$	Manber & Myers 1993; Larsson & Sadakane 1999 – prefix doubling
...and better	$O(m)$	Kärkkäinen & Sanders 2003; Ko & Aluru 2003 – the next section

The quadratic rows are the point: sorting m suffixes is not sorting m items, because the items share their characters. Every good algorithm here exploits that overlap rather than fighting it.

The skew algorithm

Kärkkäinen & Sanders, 2003 – a suffix array in $O(m)$



Interlude: radix sort

An $O(N)$ -time sort for N items whose keys split into a constant number of digits. We will need it twice.

Bucket by the least significant digit, flatten, repeat with the next digit.



after pass 1 (last digit): 100 042 892 123 333 236 777

- Each pass is one scan of the N items, so the cost is $O(N)$ per digit.
- The number of passes is the number of digits, which is a constant.
- Crucially, each pass is **stable** – items already in order for the lower digits stay that way.

Each pass is $O(N + |\Sigma|)$, so as long as the alphabet is $O(N)$ – which it is inside the DC3 recursion – a list of triples sorts in $O(N)$ with three passes. That is the whole reason DC3 is linear.

The skew algorithm: the idea

- Split the positions of T into three groups by $i \bmod 3$.
- Sort the suffixes starting in groups **1** and **2** – two thirds of them – by **recursing** on a shorter string.
- Use those ranks to sort group **0** cheaply, without recursing.
- Merge the two sorted sets.

$$T(m) = O(m) + T(2m/3)$$

Why thirds and not halves? Because of step 3. For any two positions i and j , shifting both by 1 or by 2 lands both of them in the sample – so any two suffixes can be compared with at most two characters plus a rank lookup. With an even/odd split no shift ever aligns an even and an odd position, and the merge has nothing to compare with.

We run it on $T = \text{yabbadabbado}$ ($m = 12$, a multiple of 3) – the worked example from Kärkkäinen and Sanders' own paper, so you can read the two side by side.

Step 0 – the sample

Colour each position by $i \bmod 3$. DC3 pads T on the right with three sentinels so that every position has a full triple, and takes the groups over **0 ... n** – so position $n = 12$, whose suffix is the empty one, is a group-0 position.

y	a	b	b	a	d	a	b	b	a	d	o	\$	\$	\$
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

positions 13 and 14 are pure padding – no suffix starts there

$$\mathbf{B}_0 = \{0, 3, 6, 9, 12\}$$

$$\mathbf{B}_1 = \{1, 4, 7, 10\}$$

$$\mathbf{B}_2 = \{2, 5, 8, 11\}$$

$\mathbf{C} = \mathbf{B}_1 \cup \mathbf{B}_2$ – the **sample positions**, 8 of the 13.

Groups 1 and 2 are the two thirds we recurse on. Group 0 is the third we get for free afterwards. (The paper writes the sentinel as 0; we write \$ because this example contains the letter o.)

Step 1 – sort the sample suffixes

Cut the sample positions into triples, in the order B_1 then B_2 . Sorting the suffixes of this sequence of triples is the same as sorting the sample suffixes.

$R =$

[abb]	[ada]	[bba]	[do\$]	[bba]	[dab]	[bad]	[o\$\$]
-------	-------	-------	--------	-------	-------	-------	---------

1 4 7 10 2 5 8 11
the position each triple starts at

- Radix sort the distinct triples – three passes, $O(m)$ time.
- Rename each triple by its rank, giving a string R' over an alphabet of at most $2m/3$ symbols.

$R' =$

1	2	4	6	4	5	3	7
---	---	---	---	---	---	---	---

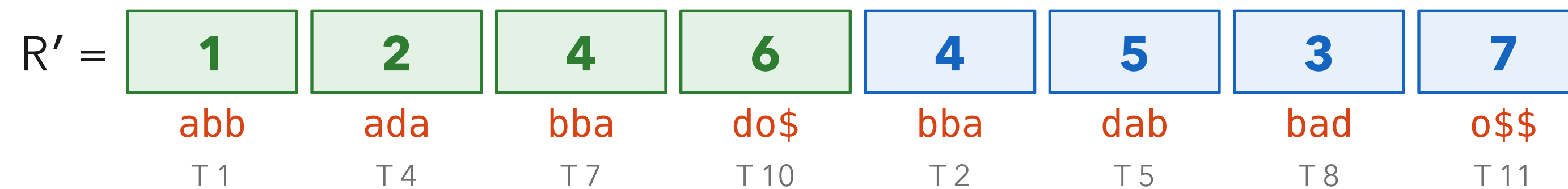
abb→1 ada→2 bad→3 bba→4 dab→5 do\$→6 o\$\$→7

If every symbol of R' were distinct we would be done – the ranks would already give the order.
Here they are not (bba appears twice), so we recurse: run the whole algorithm on R' .

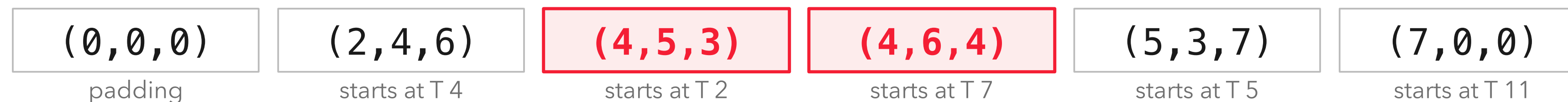
Step 1 – one level down: the recursion's own triples

The recursive call runs the same algorithm on R' – and every symbol of R' already stands for three characters of T , so a triple of them stands for nine.

each symbol of R' is one triple of T



its triples, radix sorted – smallest first



tie

decides

S2



S7



$dab < do\$ \Rightarrow S2 < S7$

The greyed cell is past the end of R' , and never matters: $do\$$ occurs once in R' , so the comparison is settled before anything reaches it.

Read the top row straight through: bba·dab·bad is bbadabbad – the first nine characters of S2.
That is what a triple of R' symbols is.

Step 1 – the ranks that come back

The recursion returns the sample suffixes in sorted order. Number them 1, 2, 3, ... – this is $\text{rank}(S_i)$, defined

only for sample positions.

sorted sample suffixes, smallest first

S1	S4	S8	S2	S7	S5	S10	S11
abbadabbado	adabbado	bado	bbadabbado	bbado	dabbado	do	o
rank 1	rank 2	rank 3	rank 4	rank 5	rank 6	rank 7	rank 8

rank(S_i)	⊥	1	4	⊥	2	6	⊥	5	3	⊥	7	8	⊥	0	0
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

⊥ marks the group-0 positions, where the rank is not yet defined; just past the end we set $\text{rank} = 0$, which is what makes the pairs in step 2 total. The recursion also returns $\text{SA}(R') = (8, 0, 1, 6, 4, 2, 5, 3, 7)$ – 9 entries for 8 symbols, because the empty suffix is included, exactly as $\text{SA}(T)$ will have 13 entries for 12 characters. Drop that first entry and the other eight are the ranks above, in order.

Everything from here on is bookkeeping with these numbers. No suffix is ever compared character by character again.

Step 2 – sort the non-sample suffixes

A group-0 suffix is one character followed by a group-1 suffix – and we already know the rank of that. So represent S_i by the pair $(T[i], \text{rank}(S_{i+1}))$ and radix sort the pairs.

suffix	its pair	what it is
S0	(y, 1)	yabbadabbado
S3	(b, 2)	badabbado
S6	(a, 5)	abbado
S9	(a, 7)	ado
S12	(\$, 0)	ε (the empty suffix)

sorted: **S12** < **S6** < **S9** < **S3** < **S0**

because $(\$, 0) < (a, 5) < (a, 7) < (b, 2) < (y, 1)$

Two radix-sort passes, $O(m)$ time, no recursion at all – the third of the positions we get for free.

Step 3 – merge

Two sorted lists, merged the ordinary way. The only question is how to compare one suffix from each – and the answer is: with ranks, in constant time.

comparing S_i (sample) with S_j (group 0)

compare as

$$i \in B_1$$

$$(T[i], \text{rank}(S_{i+1})) \text{ vs } (T[j], \text{rank}(S_{j+1}))$$

$$i \in B_2$$

$$(T[i], T[i+1], \text{rank}(S_{i+2})) \text{ vs } (T[j], T[j+1], \text{rank}(S_{j+2}))$$

Why two rules

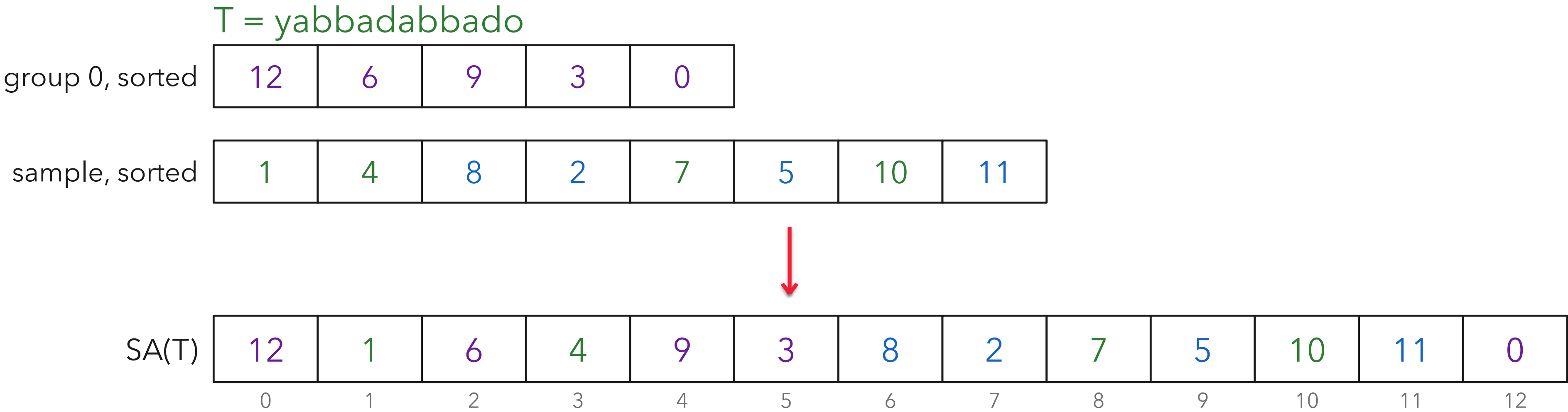
- If $i \in B_1$ then $i+1 \in B_2$ and $j+1 \in B_1$ – both sample positions, so both ranks exist.
- If $i \in B_2$ then $i+1 \in B_0$, whose rank we do not have. Take one more character: $i+2 \in B_1$ and $j+2 \in B_2$, and both of those ranks exist.

$$S1 < S6 \text{ because } (a, 4) < (a, 5) \quad S3 < S8 \text{ because } (b, a, 6) < (b, a, 7)$$

Every comparison is between fixed-size tuples of numbers, so the merge is $O(m)$. Nothing in it looks at more than three characters.

The result

Merging the two lists gives the suffix array of T.



Verified at build time against a brute-force sort of all 13 suffixes, and against the array printed in the paper.

step	what it does	cost
0 + 1	sample triples, radix sort, rename, recurse	$O(m) + T(2m/3)$
2	group-0 suffixes as pairs, radix sort	$O(m)$
3	merge the two sorted lists	$O(m)$

Why it is linear

$$T(m) = O(m) + T(2m/3)$$

|
the sort and merge

|
the recursive call,
on two thirds of the input

- Expand the O: $T(m) \leq cm + T(2m/3)$ for some constant c .
- Guess $T(m) \leq 3cm$, and assume it holds for everything smaller than m .
- Then $T(m) \leq cm + 3c(2m/3) = cm + 2cm = 3cm$. $\square$

The geometric series is what saves us: the work at the top level is $O(m)$, the next is $O(2m/3)$, the next $O(4m/9)$, and $m + 2m/3 + 4m/9 + \dots = 3m$. Linear total work, even though there are $\Theta(\log m)$ levels.

Kärkkäinen & Sanders, "Simple linear work suffix array construction", ICALP 2003; J. ACM 53(6):918-936, 2006.

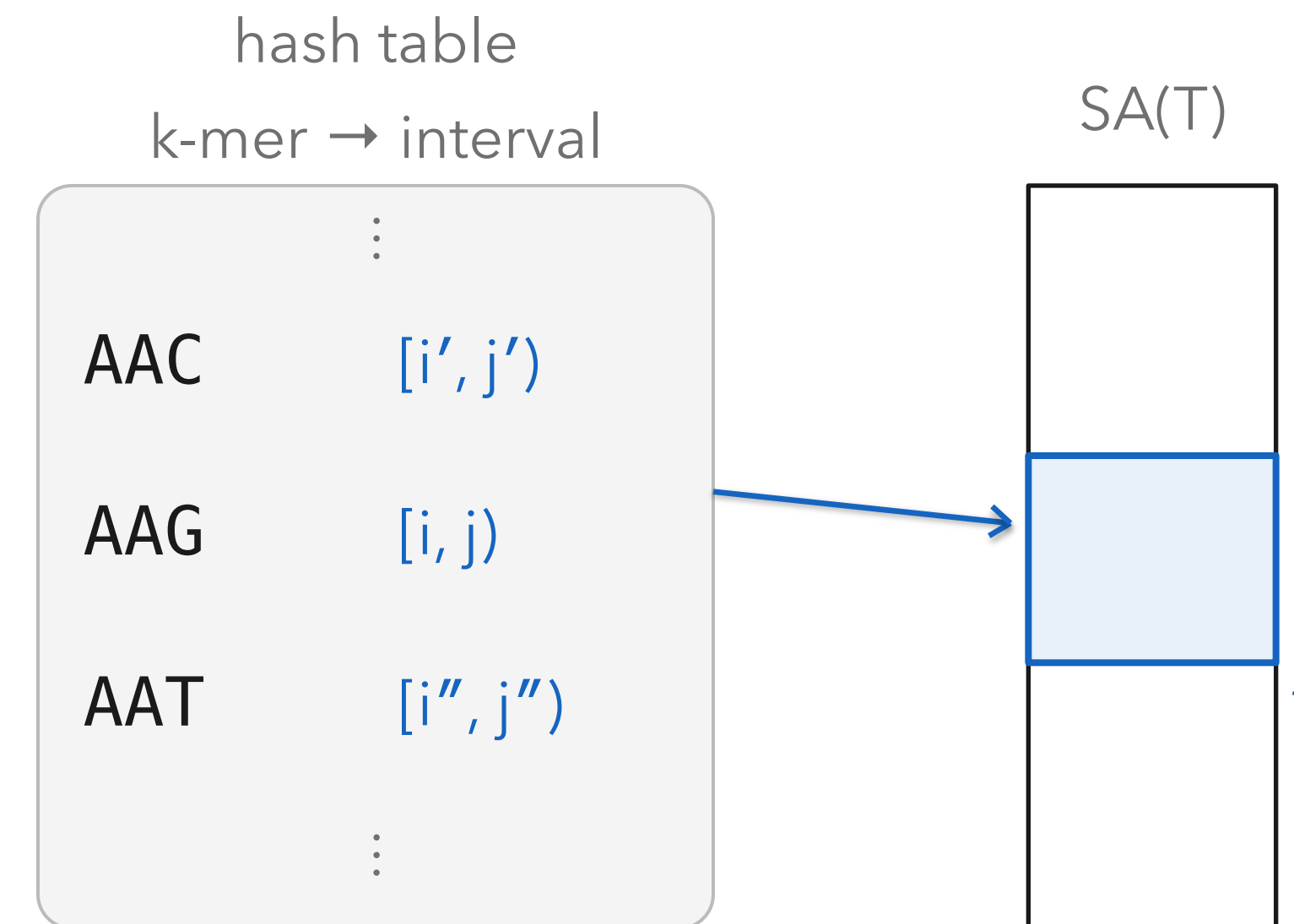
In practice

what people actually build



A practical speedup: a k-mer lookup table

- Suppose you will never search for a pattern shorter than k – in a genome, 4-mers carry no information anyway.
- Precompute, for every k-mer, the SA interval of suffixes starting with it.
- Any query of length $\geq k$ begins with some k-mer, so look that up first and binary search only inside the interval it names.



Query cost becomes **$O(n + \log(j - i))$** instead of $O(n + \log m)$ – and $j - i$ is typically tiny.

The cost is the table: 4^k intervals. At $k = 12$ that is 16.7 M entries – a few hundred MB, which for a genome-scale index is a good trade, and why aligners pick k around 10-14.

How much does that buy?

Search-speed multiples relative to a plain suffix array, over five standard corpora, for patterns of length 16.

	dna	english	proteins	sources	xml
plain SA	1.00	1.00	1.00	1.00	1.00
2-character lookup table	1.13	1.34	1.36	1.43	1.35
3-character lookup table	1.17	1.49	1.61	1.65	1.47
k-mer hash	3.75	2.88	2.70	2.90	2.03

Grabowski & Raniszewski, "Two simple full-text indexes based on the suffix array", 2014.

Note which way round it goes. Two or three DNA characters barely narrow anything – 64 buckets over millions of suffixes – so DNA gains the LEAST from the small lookup tables. With a full k-mer hash it gains the most, because a four-letter alphabet means almost every k-mer really occurs, so the table is dense and every bucket is small.

Other ideas in the same spirit

All of them trade a little space, or a little preprocessing, for fewer cache misses per query.

idea	what it changes
a k-ary (B-tree) layout of the array	one cache line holds k comparison keys instead of one, so a search touches $\log_k m$ cache lines instead of $\log_2 m$
a lookup table keyed by fixed-length Huffman codes	variable-length prefixes hash to fixed-width keys, so the table stays dense on skewed alphabets
galloping / doubling instead of bisection	finds a nearby boundary in $O(\log d)$ for a distance d, which beats $\log m$ when consecutive queries are similar

Kowalski, Grabowski et al., "Suffix arrays with a twist", arXiv:1607.08176, 2016.

Enhanced suffix arrays

With a few auxiliary tables, a suffix array can do everything a suffix tree can – at suffix-tree asymptotics and suffix-array constants.

table	size	what it is	you have already seen it
suftab	4m bytes	the suffix array itself	yes – all lecture
lcptab	m bytes	the LCP1 array	yes – the LCP section
childtab	m bytes	which SA intervals nest inside which	the tree's shape, as intervals
suflink	2m bytes	suffix links, as interval jumps	–
bwttab	m log $ \Sigma $ bits	the Burrows-Wheeler transform of T	next lecture

childtab is the key one: it records, for each SA interval, the intervals nested directly inside it – which is precisely the parent/child structure of the suffix tree, stored as pairs of integers rather than pointers. That is how an array recovers a tree.

Abouelhoda, Kurtz & Ohlebusch, "Replacing suffix trees with enhanced suffix arrays", J. Discrete Algorithms 2(1):53–86, 2004.

...and compressed suffix arrays

- **Compressed suffix arrays** store the same order in space closer to the entropy of T than to $4m$ bytes, at the cost of slower access to individual entries.
- **Compressed enhanced suffix arrays** aim for both: near-entropy space with the operations of an enhanced suffix array, at query times like $O(n \log|\Sigma| + k)$ – note that this is independent of m .

The trend across the whole lecture: the same information – the order of the suffixes – stored in progressively less space, with progressively more arithmetic needed to read it back.

Grossi & Vitter, SIAM J. Computing 35(2):378-407, 2005.

Ohlebusch & Gog, SPIRE 2009.

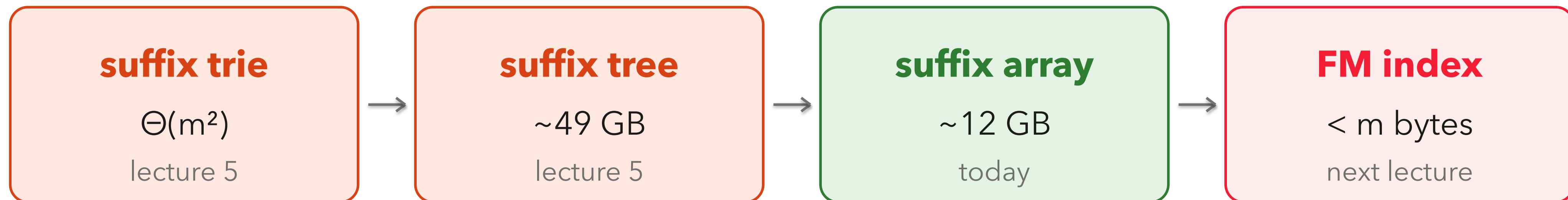
Summary

	suffix tree	suffix array	SA + LCP tables
what it stores	a tree over T's suffixes	the order of T's suffixes	the order, plus 2m LCPs
space, human genome	~49 GB	~12 GB	~37 GB
is P a substring?	$O(n)$	$O(n \log m)$	$O(n + \log m)$
report k occurrences	$O(n + k)$	$O(n \log m + k)$	$O(n + \log m + k)$
build time	$O(m)$, Ukkonen	$O(m)$, skew / DC3	$O(m)$, on the same pass

- The suffix array is the suffix tree's leaf order; LCP1 is where its leaves meet. Together they carry everything the tree's shape carried.
- Binary search alone costs a factor of $\log m$. LCPs win it back, because a suffix between two others must agree with both.
- Construction is linear without ever building a tree: split by $i \bmod 3$, recurse on two thirds, merge.

Where this goes next

Same information, progressively less space.



- The suffix array threw away the tree's pointers and kept its order. The FM index goes further: it stores a permutation of T itself and searches inside it, backwards, without storing any suffixes at all.
- It needs the Burrows-Wheeler transform – the last column of the same sorted table we built today – which is why the suffix array is the right place to have stopped.
- **Every modern read aligner uses one of the two.**